

CLIMATE CHANGE MODELING

```
from IPython.display import display, HTML
```

```
display(HTML("""
```



```
"""))
```

Objective

The objective of this project is to develop a machine learning-based model to analyze and predict the impact of various factors contributing to climate change. By leveraging historical environmental data, the model aims to classify trends, forecast future climate patterns, and support decision-making for climate policy and sustainability. The project also seeks to raise awareness by visualizing key insights and making accurate predictions that can guide environmental action.

Scope

- **Data Collection & Preprocessing:** Gathering historical data on temperature, CO₂ emissions, sea level, deforestation, and other environmental indicators.
- **Feature Engineering:** Selecting and transforming relevant variables that influence climate patterns.
- **Model Development:** Building predictive models to forecast future climate trends based on current and past data.

- Visualization & Interpretation: Creating graphs, heatmaps, and dashboards to visualize trends and support interpretation of results.
- Use Case Exploration: Demonstrating how the model can assist in climate risk assessment, policy-making, and sustainability efforts.

Features

- Sentiment Classification of Climate Texts: Accurately detects positive, neutral, and negative sentiments in climate-related content using machine learning.
- State-of-the-Art NLP Models: Utilizes advanced transformer-based language models (e.g., BERT, RoBERTa) for highly accurate and context-aware text analysis.
- Interactive Visualizations: Delivers clear and insightful visualizations, such as sentiment distribution charts and time-series plots to highlight trends.
- Robust Modeling Approach: Combines pre-trained and fine-tuned models to ensure top-tier accuracy, generalization, and reliability across datasets.

```
In [4]: import pandas as pd
import numpy as np
import plotly.express as px

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings("ignore", category=FutureWarning)

from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [5]: pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (1.6.1)
Requirement already satisfied: numpy>=1.19.5 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (2.2.4)
Requirement already satisfied: scipy>=1.6.0 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (1.15.2)
Requirement already satisfied: joblib>=1.2.0 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from scikit-learn) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 24.2 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [23]: import pandas as pd

df = pd.read_csv(r"C:\Users\Kartikey_\climate_nasa.csv.csv")
```

Display the first few rows and describing

```
In [26]: print(df.head())
print(df.describe())
```

```

      date likesCount \
0  2022-09-07T17:12:32.000Z      2
1  2022-09-08T14:51:13.000Z      0
2  2022-09-07T17:19:41.000Z      1
3  2022-09-08T00:51:30.000Z      4
4  2022-09-07T19:06:20.000Z     16

      profileName commentsCount \
0  4dca617d86b3fdce80ba7e81fb16e048c9cd9798cdfd6d...      NaN
1  518ab97f2d115ba5b6f03b2fba2ef2b120540c9681288b...      NaN
2  d82e8e24eb633fd625b0aef9b3cb625cfb044ceb8483e1...      3.0
3  37a509fa0b5177a2233c7e2d0e2b2d6916695fa9fba3f2...      NaN
4  e54fbbd42a729af9d04d9a5cc1f9bbfe8081a31c219ecb...     26.0

      text
0  Neat comparison I have not heard it before.\n ...
1  An excellent way to visualise the invisible! T...
2  Does the CO2/ghg in the troposphere affect the...
3  excellent post! I defo feel the difference - o...
4  Yes, and carbon dioxide does not harm the Eart...

      likesCount commentsCount
count  522.000000      244.000000
mean    4.720307      8.696721
std    12.053556     12.266176
min      0.000000      1.000000
25%      0.000000      2.000000
50%      1.000000      5.000000
75%      4.000000     10.000000
max    126.000000     93.000000
```

```
In [28]: print("Dataset Preview:")
display(df.head())
```

Dataset Preview:

	date	likesCount	profileName	comment
0	2022-09-07T17:12:32.000Z	2	4dca617d86b3fdce80ba7e81fb16e048c9cd9798cdfd6d...	
1	2022-09-08T14:51:13.000Z	0	518ab97f2d115ba5b6f03b2fba2ef2b120540c9681288b...	
2	2022-09-07T17:19:41.000Z	1	d82e8e24eb633fd625b0aef9b3cb625cfb044ceb8483e1...	
3	2022-09-08T00:51:30.000Z	4	37a509fa0b5177a2233c7e2d0e2b2d6916695fa9fba3f2...	
4	2022-09-07T19:06:20.000Z	16	e54fbbd42a729af9d04d9a5cc1f9bbfe8081a31c219ecb...	

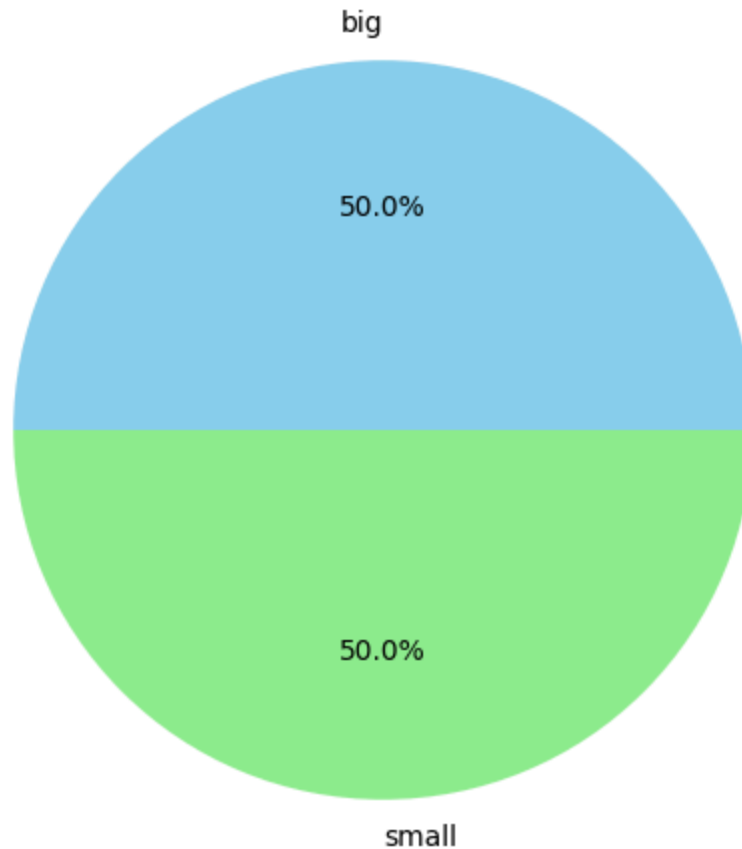
Calculate the number of characters or words in each comment, and categorize them into 'big' or 'small' groups depending on length.

```
In [31]: # Calculate the length of each text comment
df['text_length'] = df['text'].astype(str).apply(len)

# Define thresholds for 'big' and 'small'
threshold = df['text_length'].median() # Median as the threshold
df['size'] = df['text_length'].apply(lambda x: 'big' if x > threshold else 'small')

size_counts = df['size'].value_counts()
plt.figure(figsize=(6, 6))
plt.pie(size_counts, labels=size_counts.index, autopct='%1.1f%%', colors=['skyblue'])
plt.title('Distribution of Big and Small Comments')
plt.show()
```

Distribution of Big and Small Comments



Detect null entries

```
In [32]: print("\nNull Value Summary:")  
         print(df.isnull().sum())
```

Null Value Summary:

date	0
likesCount	0
profileName	0
commentsCount	278
text	18
text_length	0
size	0

dtype: int64

```
In [33]: print("\nDataset Statistics:")  
         print(df.describe())
```

Dataset Statistics:

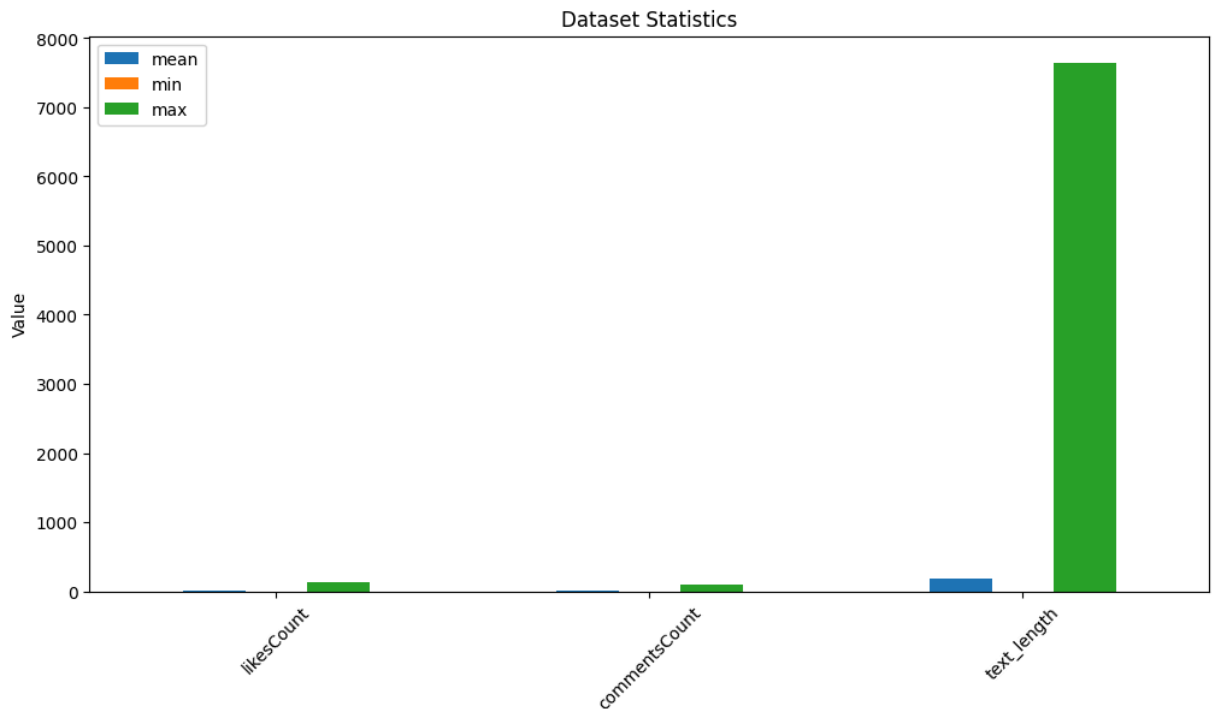
	likesCount	commentsCount	text_length
count	522.000000	244.000000	522.000000
mean	4.720307	8.696721	179.141762
std	12.053556	12.266176	519.944862
min	0.000000	1.000000	1.000000
25%	0.000000	2.000000	37.000000
50%	1.000000	5.000000	88.500000
75%	4.000000	10.000000	176.000000
max	126.000000	93.000000	7649.000000

Generate statistical plots

```
In [34]: stats = df.describe().transpose()

stats_to_plot = stats[['mean', 'min', 'max']]

stats_to_plot.plot(kind='bar', figsize=(12, 6))
plt.title('Dataset Statistics')
plt.ylabel('Value')
plt.xticks(rotation=45)
plt.show()
```

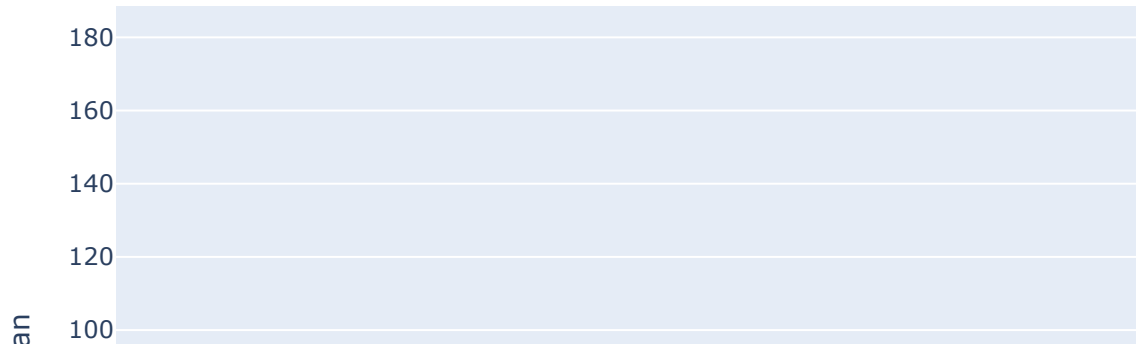


Column-wise mean statistics

```
In [35]: stats_to_plot = stats['mean'].reset_index()
stats_to_plot.columns = ['Feature', 'Mean']
```

```
fig = px.bar(stats_to_plot, x='Feature', y='Mean', title='Mean Statistics per Feature')
fig.show()
```

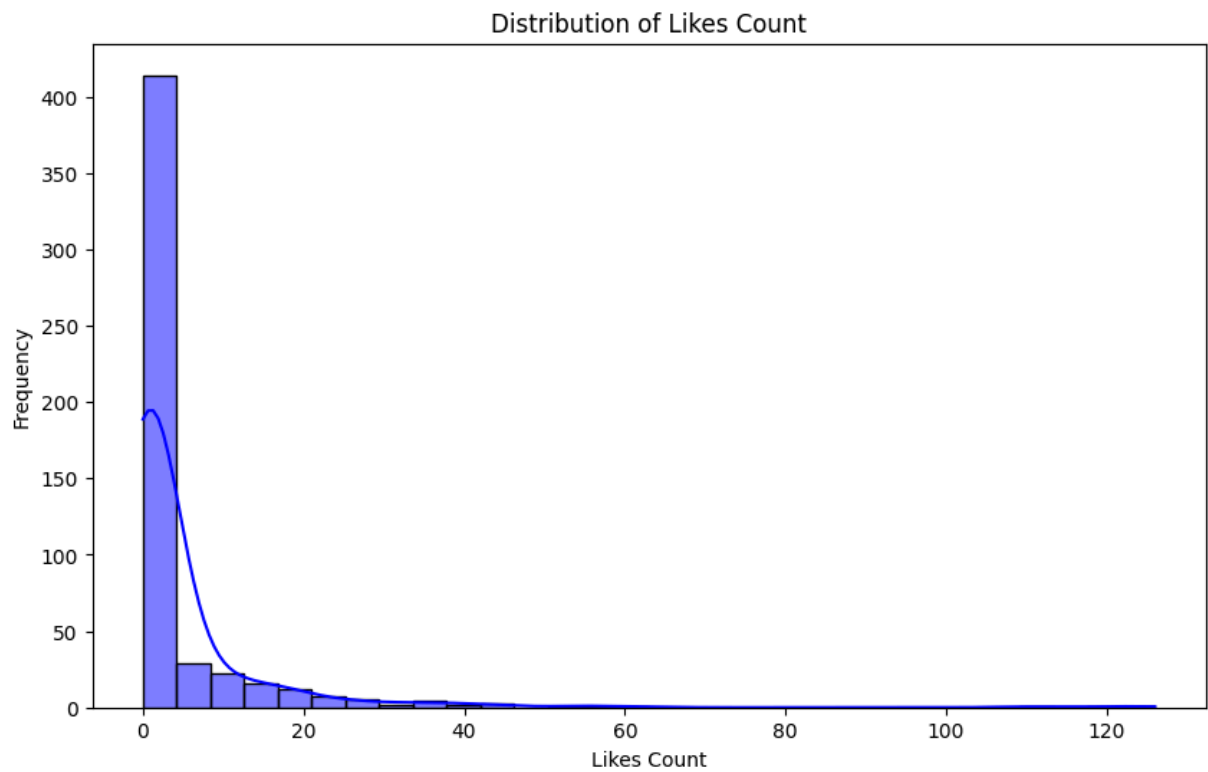
Mean Statistics per Feature



Perform Exploratory Data Analysis (EDA)

In [37]:

```
plt.figure(figsize=(10, 6))
sns.histplot(df['likesCount'], bins=30, kde=True, color='blue')
plt.title("Distribution of Likes Count")
plt.xlabel("Likes Count")
plt.ylabel("Frequency")
plt.show()
```



```
In [38]: print(df.columns)
```

```
Index(['date', 'likesCount', 'profileName', 'commentsCount', 'text',  
      'text_length', 'size'],  
      dtype='object')
```

```
In [39]: pip install wordcloud
```

Collecting wordcloud

Downloading wordcloud-1.9.4-cp313-cp313-win_amd64.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.6.1 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from wordcloud) (2.2.4)
Requirement already satisfied: pillow in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from wordcloud) (11.1.0)
Requirement already satisfied: matplotlib in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from wordcloud) (3.10.1)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (1.3.1)
Requirement already satisfied: cyclor>=0.10 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (24.2)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)
Downloading wordcloud-1.9.4-cp313-cp313-win_amd64.whl (300 kB)
Installing collected packages: wordcloud
Successfully installed wordcloud-1.9.4
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.2 -> 25.0.1

[notice] To update, run: python.exe -m pip install --upgrade pip

```
In [41]: from wordcloud import WordCloud
import matplotlib.pyplot as plt

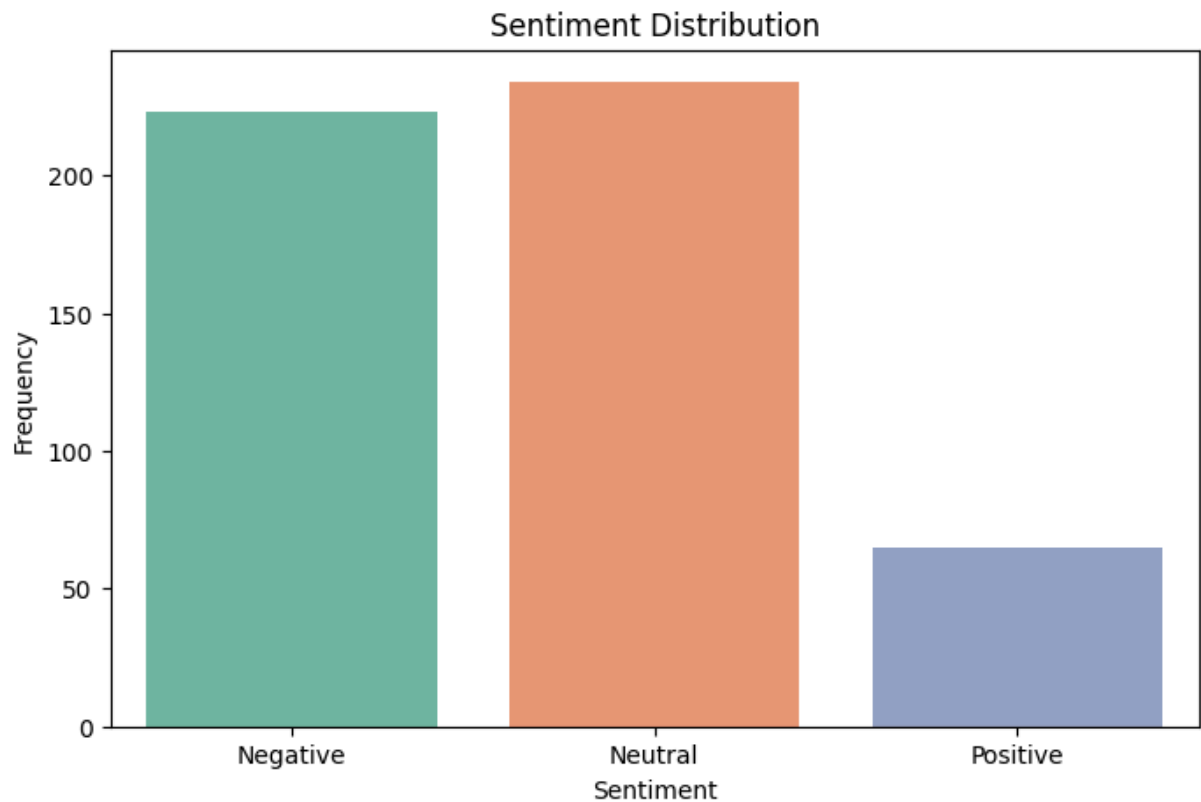
all_text = " ".join(str(comment) for comment in df['text'].dropna())

# I am gonna Generate the word cloud

wordcloud = WordCloud(width=800, height=400, background_color='white').generate(all

plt.figure(figsize=(10, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Word Cloud of Comments")
plt.show()
```


object



Text Preprocessing

```
In [44]: print(df['text'].isnull().sum())
```

18

```
In [45]: df['text'] = df['text'].fillna('')
```

Processing missing data in the 'Text' column

```
In [46]: df['text'] = df['text'].fillna('')
```

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')  
X = tfidf_vectorizer.fit_transform(df['text']).toarray()
```

```
In [47]: # Check for missing values after filling
```

```
missing_before = df['text'].isna().sum() # Count missing values before handling  
df['text'] = df['text'].fillna('') # Handle missing values  
missing_after = df['text'].isna().sum() # Count missing values after handling
```

```
In [48]: print("Missing before handling:", missing_before)
print("Missing after handling:", missing_after)
```

Missing before handling: 0
Missing after handling: 0

```
In [49]: missing_summary = df.isna().sum()
print("Missing values in each column:\n", missing_summary)
```

Missing values in each column:

date	0
likesCount	0
profileName	0
commentsCount	278
text	0
text_length	0
size	0
Sentiment	0

dtype: int64

TESTING Text Preprocessing

```
In [50]: import re

df['text'] = df['text'].apply(lambda x: re.sub(r'^\w\s', '', x.lower()))
```

```
In [53]: pip install nltk
```

Collecting nltk
 Downloading nltk-3.9.1-py3-none-any.whl.metadata (2.9 kB)
Collecting click (from nltk)
 Downloading click-8.1.8-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: joblib in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from nltk) (1.4.2)
Collecting regex>=2021.8.3 (from nltk)
 Downloading regex-2024.11.6-cp313-cp313-win_amd64.whl.metadata (41 kB)
Collecting tqdm (from nltk)
 Downloading tqdm-4.67.1-py3-none-any.whl.metadata (57 kB)
Requirement already satisfied: colorama in c:\users\kartikay_\appdata\local\programs\python\python313\lib\site-packages (from click->nltk) (0.4.6)
Downloading nltk-3.9.1-py3-none-any.whl (1.5 MB)
----- 0.0/1.5 MB ? eta -:-:--
----- 1.3/1.5 MB 8.5 MB/s eta 0:00:01
----- 1.5/1.5 MB 7.8 MB/s eta 0:00:00
Downloading regex-2024.11.6-cp313-cp313-win_amd64.whl (273 kB)
Downloading click-8.1.8-py3-none-any.whl (98 kB)
Downloading tqdm-4.67.1-py3-none-any.whl (78 kB)
Installing collected packages: tqdm, regex, click, nltk
Successfully installed click-8.1.8 nltk-3.9.1 regex-2024.11.6 tqdm-4.67.1
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 24.2 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```
In [54]: import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Download necessary NLTK resources (run once)
import nltk
nltk.download('stopwords')
nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    # Lowercase the text
    text = text.lower()
    # Remove special characters and numbers
    text = re.sub(r'^\w\s', '', text)
    text = re.sub(r'\d+', '', text)
    # Tokenize and remove stop words
    tokens = text.split()
    tokens = [word for word in tokens if word not in stop_words]
    # Lemmatize tokens
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    return ' '.join(tokens)

df['cleaned_text'] = df['text'].apply(preprocess_text)
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Kartikey\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\Kartikey\AppData\Roaming\nltk_data...
```

Text Vectorization

```
In [56]: from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
X = tfidf_vectorizer.fit_transform(df['cleaned_text']).toarray()

# Check the shape of the resulting array
print("TF-IDF Matrix shape:", X.shape)
```

TF-IDF Matrix shape: (522, 2687)


```

label_encoder = LabelEncoder()
y_train = label_encoder.fit_transform(y_train)
y_test = label_encoder.transform(y_test)

# Continue with your model training and evaluation
classifier = LogisticRegression(max_iter=1000, random_state=42)
classifier.fit(X_train, y_train)

# Make predictions
y_pred = classifier.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")

# Classification report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=[str(cls) for cls in label_encoder.classes_], # <-- fix applied h
    zero_division=0
))

```

Model Accuracy: 0.99

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	1.00	104
1	0.00	0.00	0.00	1
accuracy			0.99	105
macro avg	0.50	0.50	0.50	105
weighted avg	0.98	0.99	0.99	105

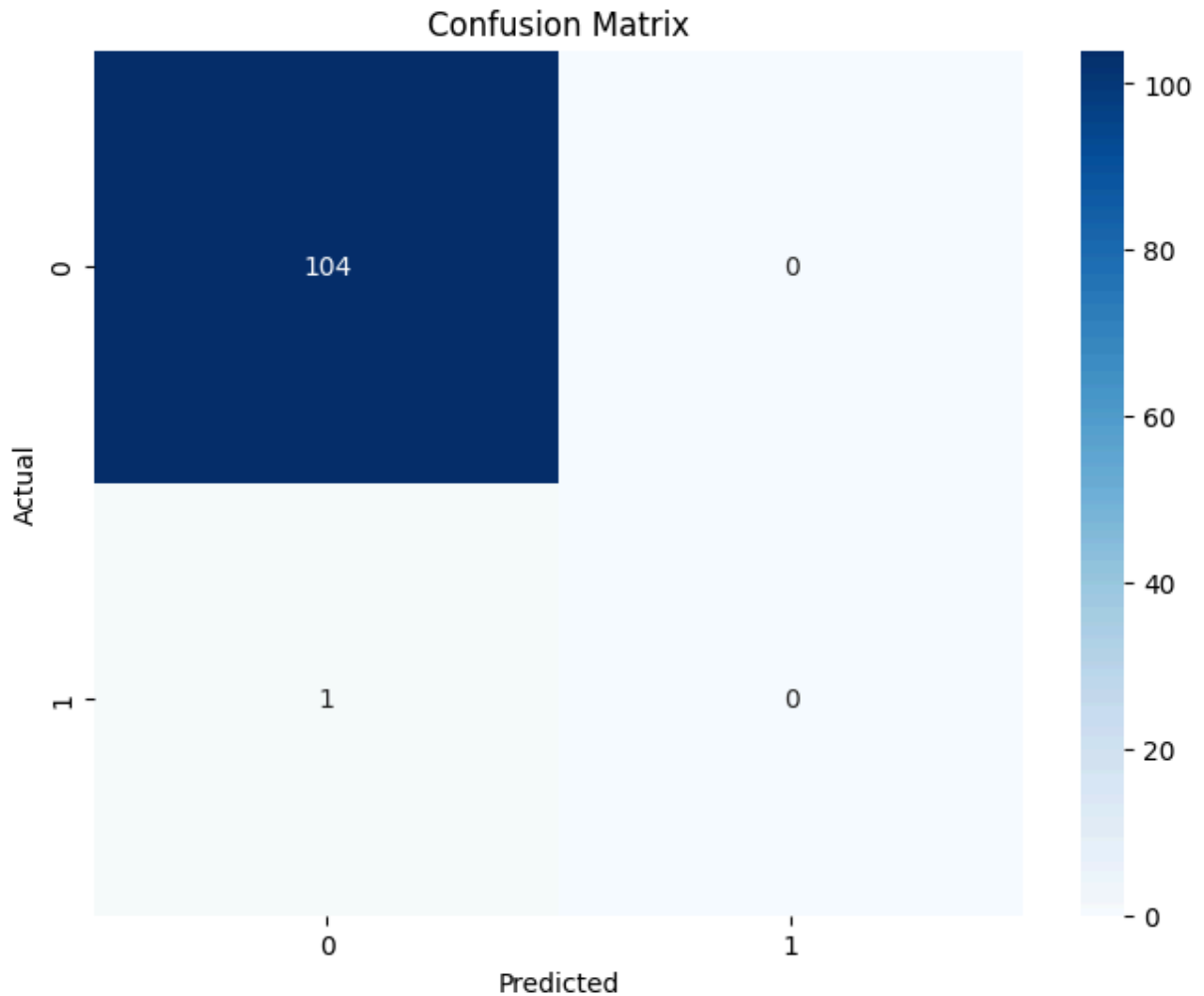
In [62]:

```

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=label_encod
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

```



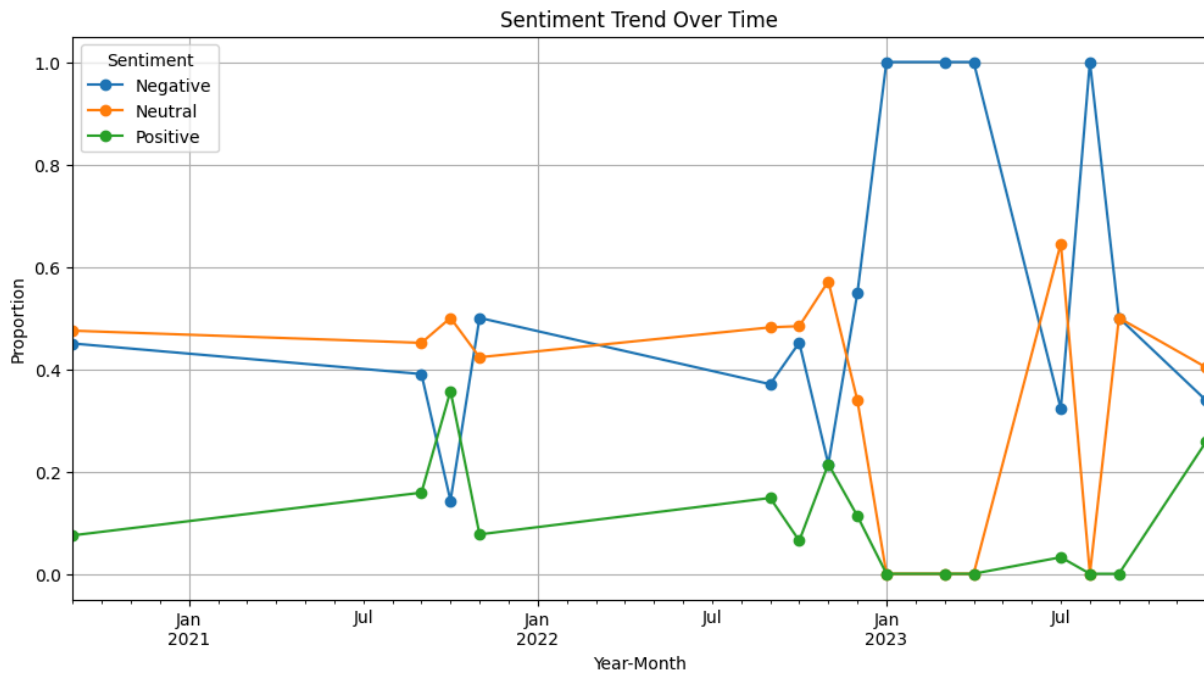
Analysis of Sentiment Trends

```
In [63]: # Convert 'date' column to datetime and remove timezone if present
df['date'] = pd.to_datetime(df['date']).dt.tz_localize(None)
df['YearMonth'] = df['date'].dt.to_period('M') # Extract Year-Month

In [64]: trend_df = df.groupby(['YearMonth', 'Sentiment']).size().unstack(fill_value=0)
trend_df = trend_df.div(trend_df.sum(axis=1), axis=0) # Convert counts to proportion

In [65]: plt.figure(figsize=(12, 6))
trend_df.plot(kind='line', figsize=(12, 6), marker='o', title="Sentiment Trend Over Time")
plt.ylabel("Proportion")
plt.xlabel("Year-Month")
plt.grid(True)
plt.legend(title="Sentiment")
plt.show()
```

<Figure size 1200x600 with 0 Axes>



Saving a the trained model

```
In [67]: import joblib

# Save the trained model and preprocessing tools
joblib.dump(classifier, "sentiment_model.pkl") # ← change 'clf' to 'classifier'
joblib.dump(tfidf_vectorizer, "tfidf_vectorizer.pkl")
joblib.dump(label_encoder, "label_encoder.pkl")

print("Model and encoders saved successfully!")
```

Model and encoders saved successfully!

Execution complete

- Model Deployment and Making Predictions on New Text data

```
In [69]: # Load the saved model, vectorizer, and Label encoder
loaded_model = joblib.load("sentiment_model.pkl")
loaded_vectorizer = joblib.load("tfidf_vectorizer.pkl")
loaded_label_encoder = joblib.load("label_encoder.pkl")

# New text data for prediction
new_texts = ["I have also seen the podcast. I congratulate you both. 🎉🎉 Many man"]

# Preprocess the text
new_texts_transformed = loaded_vectorizer.transform(new_texts)
```

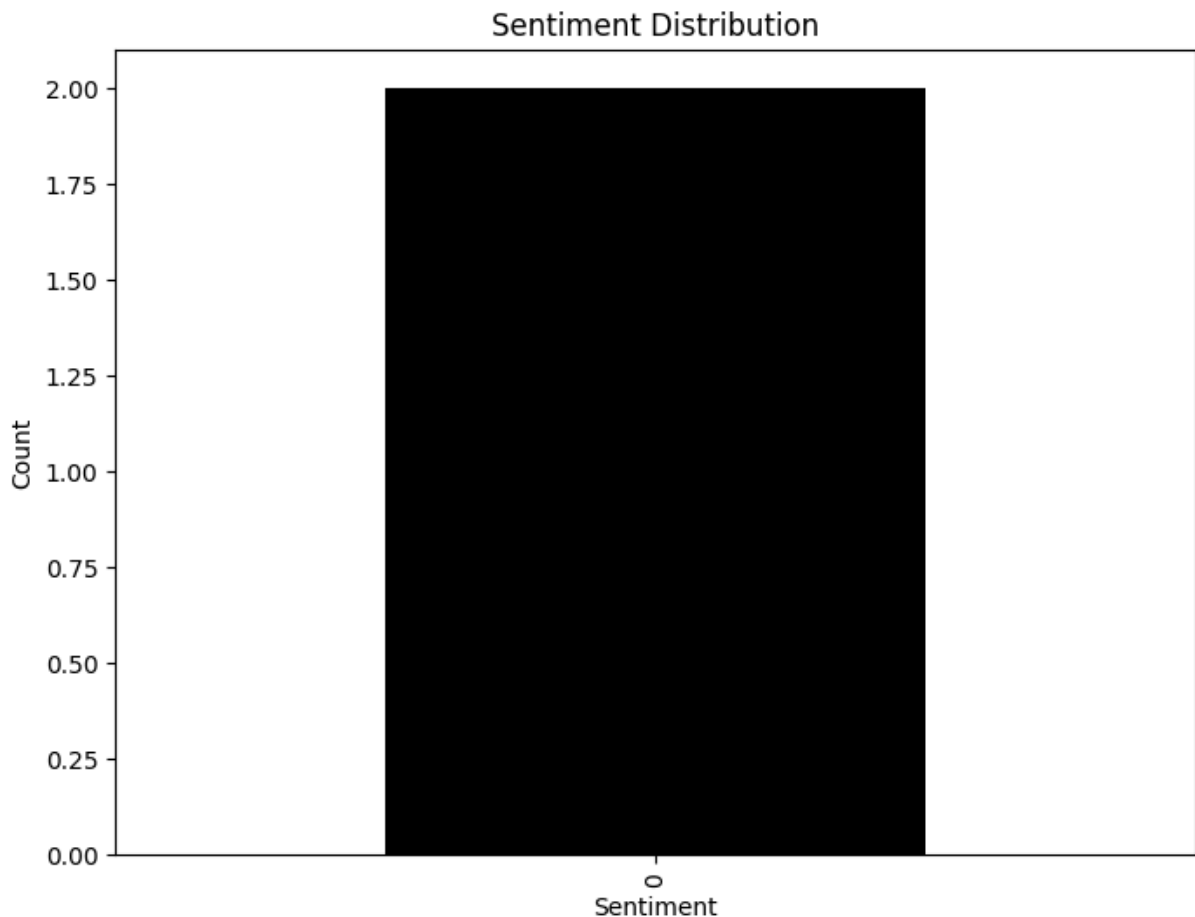
```
# Predict using the loaded model
predictions = loaded_model.predict(new_texts_transformed)

# Decode the predicted labels
decoded_predictions = loaded_label_encoder.inverse_transform(predictions)
print("Predicted Sentiments:", decoded_predictions)
```

Predicted Sentiments: [0 0]

```
In [70]: sentiments = ['0', '0']
sentiment_counts = pd.Series(sentiments).value_counts()

sentiment_counts.plot(kind='bar', color=['black', 'green'], figsize=(8, 6))
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Count")
plt.show()
```



```
In [71]: # Get probabilities for each class
probabilities = loaded_model.predict_proba(new_texts_transformed)
for i, text in enumerate(new_texts):
    print(f"Text: {text}")
    print(f"Predicted: {decoded_predictions[i]}, Probability: {probabilities[i]}")
```

Text: I have also seen the podcast. I congratulate you both. 🎉🎉 Many many congratulations & best wishes 🎉🎉🎉🎉 May God bless you both. 🎉
Predicted: 0, Probability: [0.99546668 0.00453332]
Text: Please visit barbeque nation , im sanatani ,but faiza is non vegetarian she might be having craving for non veg meal .. its perfect place to have good variety of food unlimited for 399 for veg and 449 for non veg with great ambience and hospitality.
Predicted: 0, Probability: [0.99522833 0.00477167]

Text preprocessing and feature extraction completed

Thank You

This project was completed independently, and I am grateful for the opportunity to explore and learn through hands-on experimentation. Working solo provided valuable experience in managing each phase of the machine learning pipeline — from data preprocessing and sentiment analysis to visualization and model evaluation. This experience has not only enhanced my technical proficiency in NLP and classification techniques but also deepened my understanding of climate-related trends and the power of data-driven insights for environmental awareness.

Prajakta Koli +91 9359988469 koliprjkt@gmail.com

[LINKEDIN](#)

In []: